

EVIDENCE

BOB-174 — Evidence pack

Revision: 2 **Last modified:** 2026-08-25T18:36:33Z **Guard:** tests/unit/api_layer/test_bob174_corrupt_hook_store.py (35 tests)

REVISION 2 — WHY THIS DOCUMENT CHANGED. The independent review found **zero defects in the shipped code** and a set of defects in *this evidence pack*: figures that did not reproduce, and mutation counts measured against an earlier revision of the guard and then presented as the final evidence. An evidence pack whose own numbers do not reproduce is a §11.4.262 failure regardless of how sound the code it describes is — so every figure below has been **re-measured against the current files** and the stale ones are corrected in place with the correction stated, never quietly overwritten. Two new guards were added in the same round (§9), closing the two properties the reviewer demonstrated were unpinned. **Change:** download-proxy/src/api/hooks.py **Decision record:** DESIGN_DECISION.md beside this file

Host discipline throughout: GOMAXPROCS=2 nice -n 19 ionice -c 3, .venv/bin/python -m pytest -p no:schemathesis.
PYTEST_DISABLE_PLUGIN_AUTOLOAD deliberately NOT used (BOB-165).

1. RED — observed against the unmodified tree, before any source change

The guard was written first and run first. **15 failed, 13 passed** — measured against the guard **as it stood at Revision 1 (28 tests)**. That scope label is part of the measurement and was missing in Revision 1;

the guard is now 35 tests, so this figure is a historical record of the original RED and is NOT a claim about the current file. The RED evidence for the two guards added in Revision 2 is in §9.

```
15 failed, 13 passed, 1 warning in 9.12s
FAILED ....:TestCorruptStoreIsNotReportedAsZeroHooks::test_get_does
FAILED ....:TestCorruptStoreIsNotReportedAsZeroHooks::test_get_does
FAILED ....:TestCorruptStoreIsNotReportedAsZeroHooks::test_get_does
FAILED ....:TestCorruptStoreIsNotReportedAsZeroHooks::test_error_de
FAILED ....:TestCreateRefusesRatherThanDestroying::test_create_is_i
FAILED ....:TestCreateRefusesRatherThanDestroying::test_create_is_i
FAILED ....:TestCreateRefusesRatherThanDestroying::test_create_is_i
FAILED ....:TestCreateRefusesRatherThanDestroying::test_the_operate
FAILED ....:TestCreateRefusesRatherThanDestroying::test_the_operate
FAILED ....:TestCreateRefusesRatherThanDestroying::test_the_operate
FAILED ....:TestCreateRefusesRatherThanDestroying::test_recoverable
FAILED ....:TestDeleteRefusesRatherThanDestroying::test_delete_does
FAILED ....:TestWriteIsAtomic::test_a_crash_mid_write_leaves_the_pi
FAILED ....:TestWriteIsAtomic::test_the_surviving_store_is_still_pa
FAILED ....:TestNonOSErrorWriteFailureIsPinned::test_a_serialisatio
```

The A2 data loss, reproduced verbatim

```
E AssertionError: 'prod-hook-0' is no longer in the hook store. A
truncated file is
E hand-repairable; an overwritten one is not. The create turned a
recoverable state
E into an unrecoverable one.
E assert 'prod-hook-0' in '[\n {\n      "hook_id":
"f71c5ebb-2d81-492b-87d8-61ed82e41549",
E   \n      "name": "bob174-new-hook", ... }\n]'
```

Three operator hooks in, one unrelated hook out. This is the item's reported chain, on the already-BOB-173-fixed tree.

The A5 corruption being manufactured, verbatim

```
E AssertionError: a write that FAILED PART-WAY left the store
truncated. The previous
E contents are gone and what remains is the exact corrupt input
that BOB-174's A1/A2
E chain then reads as 'no hooks' and overwrites.
E assert '[{"hook_id": "half-writ" == '[\n {\n      "hook_id":
"seeded-0", ...'
```

[{"hook_id": "half-writ — the fragment left by a failure mid-serialisation. That fragment is precisely the A1 input. The chain closes on itself, which is why the three defects are one item.

RED members that passed pre-fix, stated honestly (§11.4.6)

Two guards pass against the broken code and do **not** capture the escape by themselves. Neither is claimed to:

- `test_the_store_survives_a_refused_delete` — pre-fix, delete raised 404 *before* reaching the write, so the file was untouched for the wrong reason. It asserts the reality half that makes the 404 a lie; `test_delete_does_not_report_not_found_for_an_unreadable_store` is the member that captured the defect.
 - `test_no_partial_or_temp_file_is_left_observable_after_a_failed_write` — pre-fix there were no temp files to leave. It pins a property of the new mechanism, not the old escape.
 - `test_dispatch_runs_no_hooks_on_a_corrupt_store` (added later, §5c) — pre-fix `_load_hooks` returned `[]`, so no hooks were dispatched then either. It pins the **fail-closed** half of the dispatch decision against a future implementation that tries to salvage partial data; its partner `test_dispatch_does_not_raise_on_a_corrupt_store` is the one that constrains the new code.
-

2. Two test bugs found and fixed during the RED→GREEN transition (§11.4.1)

Both were **instrument** defects, not product defects, and both are recorded because one of them was dangerous.

(a) An impossible assertion.

`test_recoverable_operator_data_is_still_recoverable` demanded all three hook names survive a 70%-truncated seed — but a truncation by definition cut `prod-hook-2` off, so the seed never contained it. No fix could deliver it. Corrected to assert on the names the seed *actually retains*.

(b) `monkeypatch.undo()` silently re-pointed the app at production state. The atomicity tests patched `json.dump`, then called `monkeypatch.undo()` to restore it mid-test. `monkeypatch` is a single function-scoped instance **shared with the harness fixture**, so `undo()` reverted *every* patch it had made — including `HOOKS_FILE`. The app was thereby re-pointed at the real `/config/download-proxy/hooks.json`, a

GET read that non-existent path, and the test failed with `assert [] == ['prod-hook-0', ...]` for a reason having nothing to do with the code under test.

This is a §11.4.201(7)(c) instrument-path defect: **had the expected value been [], it would have passed vacuously while reading production state**. Replaced with a self-restoring one-shot patch, so no `undo()` is needed and the footgun is gone from the file. The reasoning is preserved in `_arm_one_shot_enospc`'s docstring so it cannot be reintroduced by a future edit.

3. GREEN — re-measured 2026-08-25 against the current files

Revision 1 reported “28 passed” here and “68 passed” for the five-file set. Neither reproduces: the guard is now 35 tests (28 at Revision 1, +3 landed later in that round, +2 in Revision 2 and +2 for the cross-device outcome test). **Corrected figures, all re-run today:**

Scope	Result
test_bob174_corrupt_hook_store.py (the guard)	35 passed
the five hook-touching files together	75 passed
tests/unit/api_layer/ (full sweep)	263 passed

The five files are:

```
tests/unit/api_layer/test_bob174_corrupt_hook_store.py
tests/unit/api_layer/test_bob173_hook_persistence_failure.py
tests/unit/api_layer/test_hooks_coverage.py
tests/unit/api_layer/test_hooks_endpoints.py
tests/unit/api_layer/test_hooks_remaining.py
```

```
ruff check download-proxy/src/api/hooks.py tests/unit/api_layer/ →
```

All checks passed!

4. Two gates the fix legitimately broke — RECONCILED, not fake-passed (§11.4.120)

The full tests/unit/api_layer/ sweep surfaced 5 failures. Both causes were investigated before anything was touched; neither was a regression in the fix.

(a) test_hooks_coverage.py — two tests asserted the defect

```
fake_path.write_text("not json")
hooks = api.hooks._load_hooks()
assert hooks == []           # <- the defect, codified
```

These two tests are the reason the swallow looked deliberate. They assert the exact behaviour the item's acceptance criterion (a) removes.

Reconciled to assert the new mechanism

(pytest.raises(HookStoreCorruptError)), with the reasoning in-source.

Note the sibling test_load_hooks_no_file was **left untouched and still passes** — that is the negative control proving MISSING still returns [].

(b) test_bob173_hook_persistence_failure.py — three DELETE tests, and a real behaviour change

BOB-173's seal_for_rewrite chmod'ed the hooks **file** to 0400 and probed with open(target, "a"). That was the correct needle while _save_hooks rewrote in place. The atomic write does not rewrite in place, so the seal stopped blocking anything and the delete write succeeded while the test expected a failure — the inverse of the §11.4.199 false-reproduction that file's own header warns about.

Measured before deciding (§11.4.6), not assumed:

```
append to a 0400 file : REFUSED (Permission denied)
os.replace over 0400  : SUCCEEDS
mkstemp in a 0500 dir : REFUSED (Permission denied)  <- the
atomic write fails here
```

This is a genuine, deliberate behaviour change and is recorded as

one: a hooks.json chmod'ed 0400 no longer refuses a write, because os.replace relinks a directory entry and never consults the target's mode. It is accepted because (i) file mode on the app's own state store is not a documented protection mechanism anywhere in this project, (ii) theme_state.py already behaves this way, and (iii) the realistic

deployment failure — the EACCES-on-/config case BOB-135 actually hit — is a **directory** permission problem and is still caught, which is why BOB-173’s *create* tests never failed.

Reconciled by sealing the DIRECTORY with the needle that matches the operation the code now performs (`_create_is_refused` — an atomic write must be able to create a new file). The resulting guard is a closer match to the code under test than the original was.

The premise is now pinned by a new test rather than left as a docstring claim:

```
TestAtomicWritePermissionSemanticsArePinned::test_a_read_only_target_file_does_not_stop_an_atomic_replace.
```

The reconciled gates still bite — verified, because a reconciled gate that stopped catching its own defect would be the §11.4.120 fake-pass in slow motion:

Re-verified 2026-08-25 against the current source — and unlike §5’s table, these two figures reproduce EXACTLY as Revision 1 recorded them. Each mutation is scoped to the file that owns the reconciled gate (that is the question §4 asks), and the failing tests are listed by name so “which gate bit” is measured rather than asserted:

Mutation	Scoped to	Baseline	Result	Tests that
restore BOB-173’s _save_hooks swallow	test_bob173_hook_persistence_failure.py	10 passed	5 failed , 5 passed	TestCreate, TestDelete
restore the A1 _load_hooks swallow	test_hooks_coverage.py	12 passed	2 failed , 10 passed	TestHook

Revision 1’s annotations were accurate too: the BOB-173 mutation does trip **all three** reconciled DELETE tests (the three `TestDeleteReportsPersistenceFailure` rows above), and the A1 mutation trips **both** reconciled coverage tests. Both reconciled gates therefore still catch the defect they were reconciled for — the §11.4.120 check that a reconciliation was not a fake-pass in slow motion.

5. Paired §1.1 mutations — every guard proven load-bearing

All ten re-measured 2026-08-25 against the current source and the current guard, by a scripted harness (`mutate.py`) that applies one mutation, runs both scopes, restores from a pristine copy and re-verifies the md5 (`cc19be67b0548e3ebf84d3bf6cc4a556`) before the next — so no mutation residue can survive into the tree (§11.4.84). Revision 1's table was measured against an EARLIER revision of the guard and never re-run against the final file; its rows summed to 28 while the guard had grown past that, which is the defect this re-measurement corrects.

Two scopes are reported per mutation, because one number alone is ambiguous. Revision 1's table silently mixed them — M6's "5 failed, 5 passed" was a *ten-test BOB-173 file*, not the 28-test guard, so the column could not be read consistently. Here, "guard file" is `test_bob174_corrupt_hook_store.py` alone (35 tests) and "five files" is that plus the four other hook-touching files (75 tests). A mutation that shows failures only in the five-file column is caught by a sibling guard rather than by this one, which is information the single column destroyed.

The harness also installs SIGTERM/SIGINT/atexit restore handlers, added after a tool-deadline SIGTERM killed an earlier run mid-mutation and left a mutated source in the working tree — caught by the md5 check (§9c).

#	Mutation (one pre-fix behaviour restored)	guard file (35)	five files (75)	Guard classes that caught it — measured
M1	<code>restore_load_hooks'</code> <code>except Exception:</code> <code>return []</code> (the A1 swallow)	13 f, 22 p	15 f, 60 p	<code>TestCorruptStoreIsNotReportedAsZeroH</code> <code>TestCreateRefusesRatherThanDestroying</code> <code>TestDeleteRefusesRatherThanDestroying</code>
M2	<code>restore</code> <code>open(HOOKS_FILE, "w")</code> in place of the atomic write (A5)	5 f, 30 p	9 f, 66 p	<code>TestAtomicWriteMechanicsArePinned,</code> <code>TestNonOSErrorWriteFailureIsPinned, Te</code>
M3	<code>narrow_save_hooks'</code> <code>outer except Exception</code> <code>-> except OSError</code>	2 f, 33 p	2 f, 73 p	<code>TestNonOSErrorWriteFailureIsPinned</code>

#	Mutation (one pre-fix behaviour restored)	guard file (35)	five files (75)	Guard classes that caught it — measurement
M4	revert VALID_EVENTS derivation -> divergent hardcoded literal	2 f, 33 p	3 f, 72 p	TestEventVocabularyCannotDrift
M5	drop create_hook's refusal; let the misread flow into the save (A2)	7 f, 28 p	7 f, 68 p	TestCreateRefusesRatherThanDestroying
M6	restore BOB-173's write-failure swallow (_save_hooks does not raise)	3 f, 32 p	9 f, 66 p	TestNonOSErrorWriteFailureIsPinned, Test
M7	restore os.path.isfile in place of os.path.exists	1 f, 34 p	1 f, 74 p	TestCorruptStoreIsNotReportedAsZeroH
M8	let dispatch_event propagate HookStoreCorruptError	2 f, 33 p	2 f, 73 p	TestDispatchFailsClosedWithoutBreakin
M9	drop dir=store_dir from mkstemp (reviewer RM3 - cross-device)	3 f, 32 p	3 f, 72 p	TestAtomicWriteMechanicsArePinned, TestTheCrossDeviceFailureIsCaughtForP
M10	drop the os.fsync, keep os.replace (reviewer RM2 - durability)	1 f, 34 p	1 f, 74 p	TestAtomicWriteMechanicsArePinned

What the re-measurement changed, and what it did not. Every mutation still bites, and no mutation survives — but three rows moved, and the movement is informative rather than cosmetic:

- **M2 rose from 3 failures to 5.** Revision 1 said M2's virtue was its *narrowness* — “breaks only the atomicity tests and nothing else”. That is no longer true and the claim is withdrawn: restoring the in-place `open(path, "w")` removes the temp file entirely, so it now also trips both new mechanics pins (there is no `mkstemp` to pass `dir=` to and no temp fd to `fsync`). The guard got *less* narrow and *more* correct — an in-place write genuinely violates more properties than Revision 1 could see.
- **M9 and M10 are the two the review demonstrated were UNGUARDED.** Against Revision 1's suite each passed with zero

failures; against this one M9 costs 3 tests and M10 costs 1. Those two rows are the entire point of Revision 2.

- **M6 reads 3 in the guard file but 9 across the five** — the clearest argument for reporting both scopes. Restoring BOB-173’s write-failure swallow is mostly caught by *BOB-173’s own* file, not by this one; Revision 1’s single ambiguous column showed “5 failed, 5 passed” and could not express that.

M3 still confirms the BOB-173 review’s finding directly: before that change, except `Exception` → `except OSError` passed the entire suite; it now costs two tests, in both scopes.

M7 and M10 each cost exactly one test, which is what a correctly-scoped, single-property guard looks like — the property is asserted once, by one assertion, and nothing else is coupled to it.

5b. One defect found by self-review, not by the reported chain

`_load_hooks` guarded with `os.path.isfile`, which is **also False for a directory** (or `fifo`, `socket`, `device node`) at the store path. Probed rather than assumed:

```
os.path.isfile(dir) : False    -> treated as MISSING, GET says 0
hooks
os.path.exists(dir) : True
open(dir) raises    : IsADirectoryError (an OSError -> already
handled)
```

So a directory at `hooks.json` was reported as “no hooks configured” — the *same* conflation the item is about, in a rarer flavour, and it would have survived a fix that kept `isfile`. Closed by changing one word to `os.path.exists` and letting it fall through to the `open`, where the existing `OSError` handler already covers it. A broken symlink stays `MISSING` under both spellings (`exists` follows links), which is correct — nothing readable is there.

Pinned by

`test_a_directory_at_the_store_path_is_not_reported_as_zero_hooks`, and M7 confirms it is load-bearing.

5c. A decision that was recorded but not guarded

`dispatch_event`’s corrupt-store behaviour (§3 of the decision record — log, dispatch nothing, do not propagate) was **decided and written down but initially left untested**. A recorded decision with no guard is

one edit away from silently reverting, and the reverting edit is an attractive one: “why is this swallowing an exception?” is exactly what a future reader will ask. Closed by

`TestDispatchFailsClosedWithoutBreakingUnrelatedRequests`, which asserts **both** halves — it does not raise (so a corrupt hooks file cannot 500 an unrelated search or download) **and** it runs no hooks (so the side effect fails closed). Each half alone is satisfiable by a wrong implementation, which is why both are asserted. M8 confirms they bite.

6. Negative controls (§11.4.201(1)) — the fix does not fail closed by failing always

Control	Assertion
<code>test_get_returns_an_empty_list_when_no_store_exists</code>	MISSING file → 200 { "hooks": [], "count": 0 }
<code>test_create_works_on_a_fresh_install</code>	MISSING file → create succeeds and persists [] on disk stays healthy (what deleting the last hook leaves)
<code>test_an_empty_json_list_is_a_healthy_store_not_a_corrupt_one</code>	VALID file → all three listed
<code>test_get_lists_the_configured_hooks</code>	VALID file → appends,

Control	Assertion
	does not replace VALID file →
test_delete_removes_only_the_named_hook	removes exactly one the ordinary not-found path is not swallowed by the corrupt-store guard
test_delete_of_an_absent_hook_still_404s_on_a_valid_store	the MISSING-is-not-CORRUPT boundary, asserted at the unit layer
test_load_hooks_no_file (pre-existing, untouched)	

The missing-file controls are the sharp ones: [] was the **correct** answer for that state, and a fix that collapsed it into the corrupt case would 500 every fresh install's hooks tab and refuse every create — worse than the defect being fixed.

7. Atomicity proof — not merely “it still writes”

TestWriteIsAtomic drives a **faithful** failure rather than a synthetic one: `json.dump` writes a partial fragment and then raises `OSError(ENOSPC)`, modelling a full disk or a kill mid-serialisation at the point that matters. Four assertions:

1. the previous store is **byte-identical** afterwards (the atomicity proof — with an in-place open the original is destroyed at open time, before the failure even occurs);
2. the store is **still parseable and still lists the original hooks** through the real API afterwards;
3. **no partial or temp file is observable** after a failed write;
4. **no temp file is left** after a successful write.

M2 confirms (1) and (2) are the load-bearing pair.

8. Honest boundaries

- **The dashboard still shows “No hooks registered” on a corrupt store.** The API is now honest; the UI is not. Fixing it needs an error callback in `loadHooks()` and a template branch, both in frontend/ — outside this item’s scope and live under a sibling stream. Recorded in `DESIGN_DECISION.md` §1 as a known gap and the natural follow-up.
- **`dispatch_event`’s corrupt-store arm is log-only.** Accepted only there, because that coroutine is awaited inline by the search and download handlers and raising would 500 unrelated capabilities. Reasoning and the conditions under which it should change: `DESIGN_DECISION.md` §3.
- **Not verified against a real ENOSPC or a real SIGKILL.** The injection models the failure faithfully but is not the syscall-level event.
- **`fsync` is pinned on the CALL, not on durability itself.**
TestAtomicWriteMechanicsArePinned asserts that `os.fsync` runs on the temp file’s fd *before* `os.replace`. It does **not** prove the bytes reached stable storage — durability is not in-process observable at all (after the replace the new contents are visible to every reader whether or not they were ever flushed), and only a real power-cut could distinguish the two. The call is asserted because it is the strongest thing that *can* be asserted here, and because the alternative is an unguarded line that a future “this looks

unnecessary” edit removes in silence — which the reviewer demonstrated by removing it against the whole suite without a single failure.

- **No live-container run.** Everything here is the real FastAPI app driven through `TestClient` over real files on real temp paths — no mocks of the code under test — but not a running qbittorrent-proxy container.
 - **A corrupt store is not now impossible.** The atomic write closes the crash-mid-write vector this code owns; a bad hand-edit or filesystem damage still produces one, which is why the read guard exists too.
 - **Export staleness — Revision 1’s statement was FALSE and is withdrawn; the exports are stale again now, for a different and stated reason.** Revision 1 asserted “I did not regenerate them”. Measured: the `.html`, `.pdf` and `.docx` twins carry mtime **2026-08-23 12:04**, which POSTDATES the `.md` they were generated from (12:00 / 12:02) — they *had* been regenerated, and the bullet claiming otherwise was wrong at the moment it was written. **Current state, measured 2026-08-25:** the Revision 2 edits to `DESIGN_DECISION.md` and `EVIDENCE.md` have re-staled all six twins, which now predate their sources. They were **deliberately not regenerated**, and the reason is checkable rather than asserted: `scripts/generate_markdown_exports.sh` is (a) itself modified in the index by a sibling stream (`git status --short` reports M), and (b) written to sweep **every** `*.md` under `docs/`, the project root and `scripts/` — so running it would regenerate exports across sibling streams’ in-flight documents, far outside this item’s file scope. The `.md` files are authoritative; regenerating the twins is owed work for whoever lands the exporter change (§11.4.106(E) — honest staleness beats a silent divergence, and beats sweeping another stream’s files).
 - **_save_hooks is still not concurrency-safe against a second writer.** `os.replace` makes each write atomic, so no reader ever sees a partial file — a real improvement — but two concurrent create requests can still interleave load→append→save and lose one hook. That is a separate defect (last-writer-wins), out of scope here, and **not** claimed to be fixed by this change.
-

9. Revision 2 — two properties the suite was blind to, now guarded

The review found no defect in the shipped code, but demonstrated that two of the atomic write's load-bearing properties were **asserted nowhere**: it removed each one and the entire suite stayed green. Both are properties of *how* the write is performed rather than of what it leaves behind, which is exactly why no outcome assertion could see them.

9a. `mkstemp(dir=...)` — a latent PRODUCTION defect the suite could not express

`_save_hooks`'s docstring said the temp file is created in the destination directory because `os.replace` is atomic only within a filesystem. That claim was load-bearing and unpinned, and the consequence is worse than the docstring implied. Measured on this host 2026-08-25:

```
tempfile.gettempdir() -> /tmp/.private/milosvasic      st_dev = 45
/dev/shm                                           st_dev = 30
os.replace(<file on 45>, <file on 30>)
-> OSError: [Errno 18] Invalid cross-device link
```

A cross-device rename does not silently become non-atomic — it **raises**, so every hook write fails. And the test suite is **structurally** unable to observe it: `pytest`'s `tmp_path` lives under `tempfile.gettempdir()`, so `mkstemp`'s default directory and the store directory are always the same filesystem under test. In the container they are not — the store is the bind-mounted `/config`, `/tmp` is the image overlay — so this defect class fails on **every** hook write in production while passing on **every** developer machine.

Closed at two layers:

Guard

```
TestAtomicWriteMechanicsArePinned::test_the_temp_file_is_created_in_the_destinati
```

Guard

TestTheCrossDeviceFailureIsCaughtForReal (2 tests)

The second stages the real condition — **nothing is simulated**: no `os.replace` is patched, no EXDEV is injected, the store is created under `/dev/shm` and the kernel decides. On this host both tests RAN (not skipped), confirmed by `-v`.

RED, observed against the un-pinned code (`dir=store_dir` removed — the reviewer's RM3), verbatim:

```
E AssertionError: creating a hook FAILED (500) with the store on
a different filesystem from
E tempfile.gettempdir(). ... Response: {"detail":"Hook was not
created: persisting the hook
E definition failed"}
ERROR api.hooks:hooks.py:245 Failed to save hooks: [Errno 18]
Invalid cross-device link:
  '/tmp/.private/milosvasic/.hooks-zwrc_rj1.json' -> '/dev/shm/
bob174-xdev-store-nahelgsk/hooks.json'
```

That is the production symptom reproduced end-to-end through the real FastAPI app — a runtime-class observable, not a source-class one (§11.4.226). With the kwarg dropped the guard file reports **3 failed, 32 passed**; restoring it returns **35 passed**, and the source was restored byte-identically with md5 re-verified (§11.4.84).

A false positive of my own, found by self-review and closed (§11.4.201(1)). The pin was first written as `assert "dir" in kwargs`. `tempfile.mkstemp(suffix, prefix, dir, text)` accepts `dir` **positionally**, so that assertion would have REFUSED a behaviourally-correct call — a false-positive refusal, precisely the failure mode the

rest of this file's guards are written to avoid, shipped inside the guard added to prevent one. Re-pinned on the property that is true regardless of call form: **where the temp file actually landed** (`os.path.dirname` of the path `mkstemp` returned). Both polarities are now proven, which is what makes it a validated guard rather than an asserted one (§11.4.107(10)):

Control	Source under test	Guard file result	Correct?
golden-TRUE	<code>dir= dropped (RM3)</code>	3 failed , 32 passed	fires — yes
golden-FALSE	<code>mkstemp(".json", ".hooks-", store_dir) — same directory, positional</code>	35 passed	does NOT fire — yes
baseline	unmodified	35 passed	—

A second assertion on the kwarg itself was drafted and then **deleted**: once the placement assertion passes it is unreachable, and a test that cannot fail is decoration (§11.4.224(C)). The kwarg is named in the failure message, where it belongs — as the likely cause, not as a gate.

9b. `os.fsync` — present, correct, and asserted nowhere

Dropping the `fsync` while keeping `os.replace` (the reviewer's RM2) passed the entire suite, because durability leaves no in-process trace: after the replace the new bytes are readable whether or not they were ever flushed. Pinned on the call — `os.fsync` must run **on the temp file's fd** and **before** `os.replace` — since that is the strongest assertion available, with the honest boundary recorded in §8.

RED observed against the un-pinned code (`fsync` removed): the `fsync` pin fails and **nothing else does**, which is what a correctly-scoped guard looks like. Per-mutation figures at current scope: §5, M10.

9c. A note on the instrument, not the product (§11.4.201(12))

Three measurement footguns bit during this round and are recorded so the next reader does not repeat them. All three share ONE shape — **a zero produced by a broken instrument, indistinguishable from a zero produced by a clean artifact** (§11.4.201(6)) — and all three were

caught by the same discipline: running a needle of the SAME SHAPE, for something known present, through the SAME path before believing any zero.

- **A line-wrapped literal is invisible to a line-oriented search.**

grep "14 call sites" reported **0** hits in DESIGN_DECISION.md for a string that was demonstrably there — it was wrapped as 14 call\nsites. The zero was caught only because a control needle of the same shape, for a phrase *known* present, also returned 0 (§11.4.201(7)(b)). Both occurrences were then found with a wrap-tolerant regex.

- **find here is bfs, and it rejects a relative -newermt.** find ...

-newermt '-90 minutes' returned **0 files modified in the last 90 minutes** on a tree being actively edited. The binary is bfs, not GNU findutils; it answers that argument with bfs: error: Invalid timestamp. on **stderr** — which a habitual 2>/dev/null swallowed, converting a hard error into a confident empty result. The fix is an absolute cutoff (-newermt "\$(date -d '90 minutes ago' '+%F %T')"), verified with a negative control (an impossible future cutoff → 0) and a positive one (a file just edited → present). Worth recording beyond this item: 2>/dev/null on a measurement command converts *tool failure* into *measured absence*, and the two are not the same finding.

- **A tool timeout can leave mutation residue in the working tree.**

The first mutation-harness run was SIGTERMed at a 10-minute tool deadline *mid-mutation*, leaving a mutated hooks.py behind — caught by the §11.4.84 md5 check, not by anything else. The harness now installs SIGTERM/SIGINT/atexit handlers that restore the pristine source before exiting, and is run detached (§11.4.89) rather than against a foreground deadline.